Bitswap 파일 전송 예제

개요

이 예제는 Bitswap 프로토콜을 사용하여 완전한 P2P 파일 전송 시스템을 구현하는 방법을 보여줍니다. 서버 모드와 클라이언트 모드로 동작할 수 있으며, libp2p 네트워크를 통해 UnixFS 파일을 안전하고 효율적으로 전송합니다.

이 예제를 통해 다음을 학습할 수 있습니다:

- libp2p 호스트 설정 및 피어 연결
- UnixFS 파일 시스템을 사용한 파일 구조화
- Bitswap 서버와 클라이언트 구현
- 실제 P2P 네트워크에서의 파일 전송

주요 개념

P2P 파일 전송 아키텍처

```
┌─────────────────┐          ┌─────────────────┐
│    파일 서버    │◄─────────┤ 파일 클라이언트 │
│                 │          │                 │
│ ┌─────────────┐ │          │ ┌─────────────┐ │
│ │UnixFS 파일  │ │          │ │   Bitswap   │ │
│ │   저장소    │ │          │ │ 클라이언트  │ │
│ └─────────────┘ │          │ └─────────────┘ │
```

```
│ ┌──────────────┐ │          │                   │

│ │   Bitswap    │ │          │                   │

│ │     서버     │ │          │                   │

│ └──────────────┘ │          │                   │

└──────────────────┘          └───────────────────┘

        │                           │

        └──────────── libp2p ───────────┘
```

핵심 구성 요소

1. **libp2p 호스트**: 네트워크 연결과 피어 통신 담당
2. **UnixFS**: 파일을 블록 단위로 구조화
3. **Bitswap**: 블록 단위 데이터 교환 프로토콜
4. **Blockstore**: 블록 데이터 저장소

완전한 구현 예제

1. 기본 설정 및 의존성

```go
package main


import (
"bytes"
"context"
```

```
    "crypto/rand"

    "flag"

    "fmt"

    "io"

    "log"

    mrand  "math/rand"

    "strconv"

    "strings"

    "time"



    bsclient  "github.com/ipfs/boxo/bitswap/client"

    bsnet  "github.com/ipfs/boxo/bitswap/network/bsnet"

    bsserver  "github.com/ipfs/boxo/bitswap/server"

    "github.com/ipfs/boxo/blockservice"

    blockstore  "github.com/ipfs/boxo/blockstore"

    chunker  "github.com/ipfs/boxo/chunker"

    offline  "github.com/ipfs/boxo/exchange/offline"

    "github.com/ipfs/boxo/files"

    "github.com/ipfs/boxo/ipld/merkledag"

    unixfile  "github.com/ipfs/boxo/ipld/unixfs/file"

    "github.com/ipfs/boxo/ipld/unixfs/importer/balanced"

    uih  "github.com/ipfs/boxo/ipld/unixfs/importer/helpers"

    "github.com/ipfs/go-cid"
```

```
"github.com/ipfs/go-datastore"

dsync  "github.com/ipfs/go-datastore/sync"

"github.com/libp2p/go-libp2p"

"github.com/libp2p/go-libp2p/core/crypto"

"github.com/libp2p/go-libp2p/core/host"

"github.com/libp2p/go-libp2p/core/peer"

"github.com/multiformats/go-multiaddr"

"github.com/multiformats/go-multicodec"

)
```

2. libp2p 호스트 생성

```
// makeHost는 지정된 포트에서 수신 대기하는 libp2p 호스트를 생성합니다

func  makeHost(listenPort  int, randseed  int64) (host.Host, error) {

var  r io.Reader

if  randseed == 0 {

r = rand.Reader

} else {

r = mrand.New(mrand.NewSource(randseed))

}


// 호스트용 키 쌍 생성

// 이 키는 유효한 호스트 ID를 얻기 위해 최소한 필요합니다
```

```
priv, _, err := crypto.GenerateKeyPairWithReader(crypto.RSA, 2048, r)
if err != nil {
return nil, fmt.Errorf("키 쌍 생성 실패: %w", err)
}

// 기본 libp2p 옵션 설정
opts := []libp2p.Option{
// 수신 대기할 주소 설정 (단순화를 위해 단일 인터페이스와 프로토콜로 제한)
libp2p.ListenAddrStrings(fmt.Sprintf("/ip4/127.0.0.1/tcp/%d", listenPort)),
libp2p.Identity(priv), // 생성된 개인키 사용
}

host, err := libp2p.New(opts...)
if err != nil {
return nil, fmt.Errorf("libp2p 호스트 생성 실패: %w", err)
}

return host, nil
}

// getHostAddress는 호스트의 완전한 멀티어드레스를 반환합니다
func getHostAddress(h host.Host) string {
// 호스트 멀티어드레스 구성
```

```
hostAddr, _ := multiaddr.NewMultiaddr(fmt.Sprintf("/p2p/%s", h.ID().String()))


// 주소를 캡슐화하여 완전한 멀티어드레스 구성
addr := h.Addrs()[0]
return addr.Encapsulate(hostAddr).String()
}
```

3. 파일 서버 구현

```
// 예제용 파일 생성 함수
func createExampleFile() ([]byte, error) {
b := strings.Builder{}
for i := 0; i <= 100000; i++ {
s := strconv.Itoa(i)
_, err := b.WriteString(s + "\n")
if err != nil {
return nil, err
}
}
return []byte(b.String()), nil
}


// startDataServer는 파일을 UnixFS로 저장하고 Bitswap 서버를 시작합니다
```

```go
func startDataServer(ctx context.Context, h host.Host) (cid.Cid, *bsserver.Serv

log.Println("파일 서버 시작 중...")

// 예제 파일 생성

fileBytes, err := createExampleFile()

if err != nil {

return cid.Undef, nil, fmt.Errorf("예제 파일 생성 실패: %w", err)

}

fileReader := bytes.NewReader(fileBytes)

log.Printf("파일 크기: %d bytes", len(fileBytes))



// 데이터스토어와 블록스토어 설정

ds := dsync.MutexWrap(datastore.NewMapDatastore())

bs := blockstore.NewBlockstore(ds)

bs = blockstore.NewIdStore(bs) // identity 멀티해시 처리



// 블록 서비스와 DAG 서비스 생성

bsrv := blockservice.New(bs, offline.Exchange(bs))

dsrv := merkledag.NewDAGService(bsrv)



// UnixFS 그래프 생성 매개변수

// 이 매개변수들은 kubo의 'ipfs add --cid-version=1 <file>'과 동일합니다

ufsImportParams := uih.DagBuilderParams{

Maxlinks: uih.DefaultLinksPerBlock, // 블록당 기본 최대 174개 링크
```

```
RawLeaves: true, // 실제 파일 바이트를 dag-pb 래퍼로 감싸지 않고 그대로 유지

CidBuilder: cid.V1Builder{ // 모든 링크에 CIDv1 사용

Codec: uint64(multicodec.DagPb),

MhType: uint64(multicodec.Sha2_256), // SHA2-256 해시 함수 사용

MhLength: -1, // 해시 함수의 기본 길이 사용 (256비트)

},

Dagserv: dsrv,

NoCopy: false,

}

// 파일을 고정 크기 256KiB 청크로 분할

ufsBuilder, err := ufsImportParams.New(chunker.NewSizeSplitter(fileReader, chunk

if  err != nil {

return  cid.Undef, nil, fmt.Errorf("UnixFS 빌더 생성 실패: %w", err)

}

// 균형 잡힌 레이아웃으로 그래프 배치

nd, err := balanced.Layout(ufsBuilder)

if  err != nil {

return  cid.Undef, nil, fmt.Errorf("UnixFS 그래프 생성 실패: %w", err)

}



log.Printf("UnixFS 파일 생성 완료, CID: %s", nd.Cid())



// Bitswap 프로토콜에서 수신 대기 시작
```

```
// 이 예제에서는 콘텐츠 라우팅(DHT, IPNI, 위임된 라우팅 등)을 사용하지 않습니다

// 가져올 피어를 알고 있기 때문입니다

n := bsnet.NewFromIpfsHost(h)

bswap := bsserver.New(ctx, n, bs)

n.Start(bswap)

log.Println("Bitswap 서버 시작 완료")

return nd.Cid(), bswap, nil

}
```

4. 파일 클라이언트 구현

```
// runClient는 지정된 피어에서 파일을 다운로드합니다

func runClient(ctx context.Context, h host.Host, c cid.Cid, targetPeer string)

log.Printf("파일 다운로드 시작, CID: %s", c)

log.Printf("대상 피어: %s", targetPeer)


// 클라이언트용 Bitswap 설정 (메모리만 사용)

n := bsnet.NewFromIpfsHost(h)

bswap := bsclient.New(ctx, n, nil, blockstore.NewBlockstore(datastore.NewNullDat

n.Start(bswap)

defer bswap.Close()


// 대상 피어를 멀티어드레스로 변환
```

```
maddr, err := multiaddr.NewMultiaddr(targetPeer)
if err != nil {
return nil, fmt.Errorf("멀티어드레스 파싱 실패: %w", err)
}

// 멀티어드레스에서 피어 ID 추출
info, err := peer.AddrInfoFromP2pAddr(maddr)
if err != nil {
return nil, fmt.Errorf("피어 정보 추출 실패: %w", err)
}

// 콘텐츠를 가진 피어에 직접 연결
// 일반적으로 이 피어는 콘텐츠 라우팅 시스템에서 제공되지만,
// go-bitswap은 연결된 피어에게도 콘텐츠를 요청하므로 이 방법도 작동합니다
log.Printf("피어 연결 중: %s", info.ID)
if err := h.Connect(ctx, *info); err != nil {
return nil, fmt.Errorf("피어 연결 실패: %w", err)
}
log.Println("피어 연결 성공")

// DAG 서비스 생성 (읽기 전용)
dserv := merkledag.NewReadOnlyDagService(
merkledag.NewSession(ctx,
```

```
merkledag.NewDAGService(
blockservice.New(
blockstore.NewBlockstore(datastore.NewNullDatastore()),
bswap))))


// 루트 노드 가져오기
log.Println("루트 노드 가져오는 중...")
nd, err := dserv.Get(ctx, c)
if err != nil {
return nil, fmt.Errorf("루트 노드 가져오기 실패: %w", err)
}
log.Println("루트 노드 가져오기 성공")


// UnixFS 파일로 변환
log.Println("UnixFS 파일 변환 중...")
unixFSNode, err := unixfile.NewUnixfsFile(ctx, dserv, nd)
if err != nil {
return nil, fmt.Errorf("UnixFS 파일 생성 실패: %w", err)
}


// 파일 데이터 읽기
log.Println("파일 데이터 읽는 중...")
var buf bytes.Buffer
```

```
if f, ok := unixFSNode.(files.File); ok {
if _, err := io.Copy(&buf, f); err != nil {
return nil, fmt.Errorf("파일 읽기 실패: %w", err)
}
}

log.Printf("파일 다운로드 완료, 크기: %d bytes", buf.Len())
return buf.Bytes(), nil
}
```

5. 메인 함수 및 실행 로직

```
const exampleBinaryName = "bitswap-transfer"

func main() {
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

// 명령줄 옵션 파싱
targetF := flag.String("d", "", "연결할 대상 피어")
seedF := flag.Int64("seed", 0, "ID 생성용 랜덤 시드 설정")
portF := flag.Int("port", 0, "수신 대기할 포트 (0이면 자동 할당)")
```

```
    flag.Parse()


    // libp2p 호스트 생성
    h, err := makeHost(*portF, *seedF)
    if err != nil {
    log.Fatal("호스트 생성 실패:", err)
    }
    defer h.Close()


    fullAddr := getHostAddress(h)
    log.Printf("내 주소: %s", fullAddr)


    if *targetF == "" {
    // 서버 모드: 파일을 호스팅하고 요청을 대기
    log.Println("서버 모드로 시작")
    c, bs, err := startDataServer(ctx, h)
    if err != nil {
    log.Fatal("데이터 서버 시작 실패:", err)
    }
    defer bs.Close()
    log.Printf("UnixFS 파일 호스팅 중, CID: %s", c)
    log.Println("인바운드 연결 및 Bitswap 요청 대기 중...")
    log.Printf("다른 터미널에서 다음 명령을 실행하세요:")
```

```
log.Printf("./%s -d %s", exampleBinaryName, fullAddr)



// 취소될 때까지 실행

<-ctx.Done()

} else {

// 클라이언트 모드: 파일 다운로드

log.Println("클라이언트 모드로 시작")

// 예제에서는 고정된 CID를 사용하지만, 실제로는 서버에서 제공받아야 합니다

// 이 CID는 0부터 100k까지의 숫자가 포함된 파일입니다

const fileCid = "bafybeiecq2irw4fl5vunnxo6cegoutv4de63h7n27tekkjtak3jrvrzzhe"

fileData, err := runClient(ctx, h, cid.MustParse(fileCid), *targetF)

if err != nil {

log.Fatal("클라이언트 실행 실패:", err)

}

log.Println("데이터 찾기 성공!")

log.Printf("파일 내용 (처음 200자): %s...", string(fileData[:min(200, len(fileData

log.Println("파일에는 0부터 100k까지의 모든 숫자가 포함되어 있습니다!")

}

}



// min 헬퍼 함수

func min(a, b int) int {

if a < b {
```

```
return a
}
return b
}
```

고급 기능 구현

1. 진행률 모니터링

```
// ProgressMonitor는 다운로드 진행률을 추적합니다
type ProgressMonitor struct {
totalBlocks int
downloadedBlocks int
totalBytes int64
downloadedBytes int64
startTime time.Time
}

func NewProgressMonitor() *ProgressMonitor {
return &ProgressMonitor{
startTime: time.Now(),
}
}
```

```
func (pm *ProgressMonitor) OnBlockReceived(blockSize int) {
pm.downloadedBlocks++
pm.downloadedBytes += int64(blockSize)
if pm.downloadedBlocks%10 == 0 { // 10블록마다 진행률 출력
elapsed := time.Since(pm.startTime)
speed := float64(pm.downloadedBytes) / elapsed.Seconds() / 1024 // KB/s
log.Printf("진행률: %d/%d 블록, %.2f KB/s",
pm.downloadedBlocks, pm.totalBlocks, speed)
}
}

// 진행률 모니터링을 포함한 클라이언트 구현
func runClientWithProgress(ctx context.Context, h host.Host, c cid.Cid, targetP
monitor := NewProgressMonitor()
// ... (기존 클라이언트 코드와 동일)
// 블록을 받을 때마다 진행률 업데이트
// 실제 구현에서는 Bitswap의 이벤트 시스템을 사용해야 합니다
return nil, nil // 구현 생략
}
```

2. 다중 파일 전송

```
// MultiFileServer는 여러 파일을 동시에 호스팅합니다
type MultiFileServer struct {
host host.Host
bswap *bsserver.Server
files  map[string]cid.Cid // 파일명 -> CID 매핑
}

func  NewMultiFileServer(ctx context.Context, h host.Host) (*MultiFileServer, er
ds := dsync.MutexWrap(datastore.NewMapDatastore())
bs := blockstore.NewBlockstore(ds)
bs = blockstore.NewIdStore(bs)

n := bsnet.NewFromIpfsHost(h)
bswap := bsserver.New(ctx, n, bs)
n.Start(bswap)

return &MultiFileServer{
host: h,
bswap: bswap,
files: make(map[string]cid.Cid),
}, nil
}
```

```go
func (mfs *MultiFileServer) AddFile(ctx context.Context, filename  string, data

// UnixFS 파일 생성 로직 (위의 startDataServer와 유사)

// ... 구현 생략

// 파일 CID를 맵에 저장

// mfs.files[filename] = fileCid

return  cid.Undef, nil  // 구현 생략

}


func (mfs *MultiFileServer) ListFiles() map[string]cid.Cid {

return  mfs.files

}


func (mfs *MultiFileServer) Close() error {

return  mfs.bswap.Close()

}
```

3. 재시도 및 오류 복구

```go
// RetryConfig는 재시도 설정을 정의합니다

type RetryConfig struct {

MaxRetries  int

BaseDelay time.Duration
```

```
MaxDelay time.Duration
}


// downloadWithRetry는 재시도 로직을 포함한 다운로드를 수행합니다
func downloadWithRetry(ctx context.Context, h host.Host, c cid.Cid, targetPeer
var lastErr error
for attempt := 0; attempt <= config.MaxRetries; attempt++ {
if attempt > 0 {
// 지수 백오프로 지연
delay := time.Duration(float64(config.BaseDelay) * math.Pow(2, float64(attempt-1
if delay > config.MaxDelay {
delay = config.MaxDelay
}
log.Printf("재시도 %d/%d, %v 후 재시도", attempt, config.MaxRetries, delay)
time.Sleep(delay)
}
data, err := runClient(ctx, h, c, targetPeer)
if err == nil {
if attempt > 0 {
log.Printf("재시도 성공 (시도 %d)", attempt+1)
}
return data, nil
}
```

```
lastErr = err
log.Printf("시도 %d 실패: %v", attempt+1, err)
}
return nil, fmt.Errorf("모든 재시도 실패, 마지막 오류: %w", lastErr)
}
```

실행 방법

1. 빌드

```
# 프로젝트 디렉토리에서
go mod init bitswap-transfer-example
go mod tidy
go build -o bitswap-transfer
```

2. 서버 실행

```
# 터미널 1에서 서버 시작
./bitswap-transfer
```

출력 예시:

```
2024/01/15 10:30:00 내 주소: /ip4/127.0.0.1/tcp/54321/p2p/QmExample...
```

```
2024/01/15 10:30:00 서버 모드로 시작

2024/01/15 10:30:00 파일 서버 시작 중...

2024/01/15 10:30:00 파일 크기: 688894 bytes

2024/01/15 10:30:00 UnixFS 파일 생성 완료, CID: bafybeiecq2irw4fl5vunnxo6cegoutv4c

2024/01/15 10:30:00 Bitswap 서버 시작 완료

2024/01/15 10:30:00 UnixFS 파일 호스팅 중, CID: bafybeiecq2irw4fl5vunnxo6cegoutv4c

2024/01/15 10:30:00 인바운드 연결 및 Bitswap 요청 대기 중...

2024/01/15 10:30:00 다른 터미널에서 다음 명령을 실행하세요:

2024/01/15 10:30:00 ./bitswap-transfer -d /ip4/127.0.0.1/tcp/54321/p2p/QmExample
```

3. 클라이언트 실행

```
# 터미널 2에서 클라이언트 시작 (서버 주소 사용)

./bitswap-transfer  -d  /ip4/127.0.0.1/tcp/54321/p2p/QmExample...
```

출력 예시:

```
2024/01/15 10:31:00 내 주소: /ip4/127.0.0.1/tcp/54322/p2p/QmClient...

2024/01/15 10:31:00 클라이언트 모드로 시작

2024/01/15 10:31:00 파일 다운로드 시작, CID: bafybeiecq2irw4fl5vunnxo6cegoutv4de63

2024/01/15 10:31:00 대상 피어: /ip4/127.0.0.1/tcp/54321/p2p/QmExample...

2024/01/15 10:31:00 피어 연결 중: QmExample...

2024/01/15 10:31:00 피어 연결 성공

2024/01/15 10:31:00 루트 노드 가져오는 중...
```

```
2024/01/15 10:31:00 루트 노드 가져오기 성공

2024/01/15 10:31:00 UnixFS 파일 변환 중...

2024/01/15 10:31:00 파일 데이터 읽는 중...

2024/01/15 10:31:01 파일 다운로드 완료, 크기: 688894 bytes

2024/01/15 10:31:01 데이터 찾기 성공!

2024/01/15 10:31:01 파일 내용 (처음 200자): 0

2024/01/15 10:31:01 파일에는 0부터 100k까지의 모든 숫자가 포함되어 있습니다!
```

단계별 구현 설명

1단계: 환경 설정

- Go 모듈 초기화 및 의존성 설치
- 필요한 패키지 임포트

2단계: libp2p 호스트 생성

- RSA 키 쌍 생성으로 고유한 피어 ID 생성
- TCP 리스너 설정으로 네트워크 연결 준비
- 호스트 인스턴스 생성 및 설정

3단계: 파일 서버 구현

- 예제 파일 생성 (0부터 100,000까지의 숫자)
- UnixFS 형식으로 파일 구조화 및 청킹
- Bitswap 서버 시작으로 요청 대기

4단계: 파일 클라이언트 구현

- 대상 피어 연결 설정
- Bitswap 클라이언트로 블록 요청
- UnixFS 파일 재구성 및 데이터 추출

5단계: 실행 및 테스트

- 서버와 클라이언트 모드 분기 처리
- 명령줄 인자 파싱 및 실행
- 성공적인 파일 전송 확인

모범 사례

1. 리소스 관리

```
// 항상 리소스를 적절히 정리
defer h.Close()
defer bswap.Close()
```

2. 오류 처리

```
// 구체적인 오류 메시지 제공
if err != nil {
return nil, fmt.Errorf("구체적인 작업 실패: %w", err)
```

```
}
```

3. 로깅

```
// 진행 상황을 명확히 로깅
log.Printf("작업 진행 중: %s", status)
```

4. 컨텍스트 사용

```
// 취소 가능한 컨텍스트 사용
ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
defer  cancel()
```

문제 해결

일반적인 문제와 해결책

1. **피어 연결 실패**

- 방화벽 설정 확인
- 포트 사용 가능 여부 확인
- 멀티어드레스 형식 검증

2. **블록 다운로드 실패**

- 네트워크 연결 상태 확인
- 서버가 실제로 해당 CID를 가지고 있는지 확인

- 타임아웃 설정 조정

3. **메모리 사용량 증가**

- 큰 파일의 경우 스트리밍 방식 사용
- 블록스토어 크기 제한 설정
- 가비지 컬렉션 최적화

4. **성능 문제**

- 청크 크기 조정
- 동시 연결 수 최적화
- 네트워크 대역폭 고려

이 예제를 통해 Bitswap을 사용한 완전한 P2P 파일 전송 시스템을 구현할 수 있으며, 실제 애플리케이션에서는 추가적인 보안, 인증, 암호화 기능을 고려해야 합니다.